

The image features a white background with decorative geometric shapes in the corners. The top right corner has a magenta triangle and a yellow triangle. The bottom right corner has a dark blue triangle and a light blue triangle.

F.U.C.K

Dev - FastAPI

Sergio Iván Galvis Motoa

F.U.C.K

What is?

- **Find:** Is a way to ask for a single or multiple registers from a data source.
- **Update:** The action to change some data in a register or the entire register.
- **Create:** Adding data in the data source.
- **Kill:** Is the action to delete or inactivate a register in the data source.

*Data source could be assumed as a persistence of info (plain text, database, volatile memory)

F.U.C.K

How to?

- **Define Model:** Is very important to create your models, it allows to identify the entities of the project.
- **Setting up:** Depending on your data persistence strategy you should setting up the DB, List, CSV File, etc.
- **Create:** Each one of the endpoints with the logic should be created for interact. (F.U.C.K.)
- **Think:** What's the way to manage the deletion or inactivation of the data in the current business logic?

*The current example works with FastAPI and SQLAlchemy.

● ● ● models.py

```
from sqlalchemy import SQLAlchemy, Field
```

```
class CustomerBase(SQLAlchemy): 3 usages
```

```
    name: str | None = Field(default=None, nullable=True)
```

```
    description: str | None = Field(default=None, nullable=True)
```

```
class Customer(CustomerBase, table=True): 13 usages
```

```
    id: int | None = Field(default=None, primary_key=True)
```

```
class CustomerCreate(CustomerBase): 2 usages
```

```
    pass
```

```
class CustomerUpdate(CustomerBase): 2 usages
```

```
    pass
```

Indicates this model will be a table

This field is the primary key

Define Model

F.U.C.K

db.py

```
from typing import Annotated
from sqlalchemy import create_engine, Session, SQLModel
from fastapi import Depends, FastAPI

db_name = "db.sqlite3"
db_url = f"sqlite:///db_name"

engine = create_engine(db_url)

def create_tables(app: FastAPI):
    SQLModel.metadata.create_all(engine)
    yield

def get_session() -> Session:
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]
```

Setting up

F.U.C.K

● ● ● main.py

```
from fastapi import FastAPI
from db import create_tables, SessionDep
from models import Customer, CustomerCreate
```

🔗 /

```
app = FastAPI(lifespan=create_tables)
```

🔗 /customers

```
@app.post(path: "/customers", response_model=Customer, status_code=201)
async def create_customer(customer_data: CustomerCreate, session: SessionDep):
    new_customer = Customer.model_validate(customer_data)
    session.add(new_customer)
    session.commit()
    session.refresh(new_customer)
    return new_customer
```

Create

F.U.C.K

● ● ● main.py

```
from fastapi import FastAPI, HTTPException
from db import create_tables, SessionDep
from models import Customer, CustomerCreate
```

🔗 /

```
app = FastAPI(lifespan=create_tables)
```

🔗 /customers/{customer_id}

```
@app.get(path: "/customers/{customer_id}", response_model=Customer)
```

```
async def find_one_customer(customer_id:int, session:SessionDep):
    customer_db = session.get_one(Customer, customer_id)
    if not customer_db:
        raise HTTPException(status_code=404, detail="Customer not found")
    return customer_db
```

Find - One

F.U.C.K

● ● ● main.py

```
from fastapi import FastAPI, HTTPException
from db import create_tables, SessionDep
from models import Customer, CustomerCreate
```

🔗 /

```
app = FastAPI(lifespan=create_tables)
```

🔗 /customers

```
@app.get(path: "/customers", response_model=list[Customer])
async def find_all_customers(session: SessionDep) -> list[Customer]:
    return session.query(Customer).all()
```

Find - All

F.U.C.K

● ● ● main.py

```
from fastapi import FastAPI, HTTPException
from db import create_tables, SessionDep
from models import Customer, CustomerCreate, CustomerUpdate
```

🔗 /

```
app = FastAPI(lifespan=create_tables)
```

🔗 /customer/{customer_id}

```
@app.patch(path: "/customer/{customer_id}", response_model=Customer)
```

```
async def update_customer(new_customer: CustomerUpdate, customer_id:int, session:SessionDep):
    customer_db = session.get_one(Customer, customer_id)
    if not customer_db:
        raise HTTPException(status_code=404, detail="Customer not found")
    customer_update = new_customer.model_dump(exclude_unset=True)
    customer_db.sqlmodel_update(customer_update)
    session.add(customer_db)
    session.commit()
    session.refresh(customer_db)
    return customer_db
```

Update

F.U.C.K

● ● ● main.py

```
from fastapi import FastAPI, HTTPException
from db import create_tables, SessionDep
from models import Customer, CustomerCreate, CustomerUpdate
```

🔗 /

```
app = FastAPI(lifespan=create_tables)
```

🔗 /customers/{customer_id}

```
@app.delete("/customers/{customer_id}")
```

```
async def kill_one_register(customer_id:int, session:SessionDep):
    customer_db = session.get_one(Customer, customer_id)
    if not customer_db:
        raise HTTPException(status_code=404, detail="Customer not found")
    session.delete(customer_db)
    session.commit()
    return {"Customer has been deleted"}
```

Kill F.U.C.K